

AN1.5 - Agent Run Persistence (A3)

Sprint: AT24 AI Agents - Agent Framework Foundation **Step:** A3 - Agent Run / Step / ToolCall / Evidence persistence **Depends on:** AN1.1-AN1.4, AF-v1 contracts, G02 (closed) **Gate:** G03 - schema + generated SQL review. **Migration NOT applied.**

Migration status: **GENERATED** / **REVIEWED** / **NOT APPLIED**.
prisma/migrations/20260906120000_add_agent_runtime_persistence/migration.sql exists and is hand-reviewed. It has not been run against any database. Applying it needs an explicit go-ahead (AN1.2 P1). Never prisma migrate dev.

1. Pre-work - existing schema inspected

Model	Decision	Why
Agent	Left untouched	AN1.2 D3 - frozen legacy layer. Adding a runs back-relation would force a @relation on Agent; instead AgentRun.agentId is a bare indexed string (below).
AgentTask / AgentMemory / AgentActivity	Left untouched	Same. New runtime writes new tables; the legacy /dashboard/agents UI keeps reading these. No cross-writes.
User	Left untouched	AgentRun.userId is a bare denormalized string, exactly like AgentTask.userId, IntelligenceAnalysisRun.userId, Message.userId. Ownership is enforced in the service layer, never trusted from client input.
IntelligenceAnalysisRun / IntelligenceAnalysisOutcome	Pattern reused	Bare userId, Json? snapshots, cascade onDelete parent->child, append-only outcome rows.
AuditLog / Message / AgentActivity	Pattern reused	Append-only, immutable, no updatedAt/deletedAt on the child tables.
AlgoTestRun (most recent migration)	Style reused	Unmapped PascalCase table, Float for numeric, Json? payloads, hand-written migration SQL, YYYYMMDDHHMMSS_name folder.

Migration workflow observed: this repo does **not** use prisma migrate dev. Migration SQL is hand-authored (or migrate diff-generated then reviewed) to match the schema, folder name is a timestamp prefix. A3 follows that exactly.

2. Schema diff

Purely additive. prisma/schema.prisma gains **6 enums + 4 models** at the end of the file (after AlgoTestRun). No existing line changes.

```

+ enum AgentRunStatus      (14 values - mirrors AF-v1 AGENT_RUN_STATUSES)
+ enum AgentRunTrigger      (manual | schedule | event | supervisor)
+ enum AgentStepKind        (8 values - mirrors AF-v1 AGENT_STEP_KINDS)
+ enum AgentStepStatus      (ok | error | skipped)
+ enum AgentToolCallStatus  (5 values - mirrors AF-v1 ToolResultStatus)
+ enum AgentEvidenceType    (10 values - mirrors AF-v1 AGENT_EVIDENCE_TYPES)

+ model AgentRun            (mutable status row; steps[]/toolCalls[]/evidence[])
+ model AgentStep           (append-only; @@unique([runId, index]))
+ model AgentToolCall       (append-only; runId + stepId FKs)
+ model AgentEvidence       (append-only; runId + stepId FKs, toolCallId FK optional)

```

2.1 Contract amendments (additive - AF-v1 unchanged)

types/agent-framework/agent-run-contract.ts:

- `AgentStep.createdAt` and `AgentToolCall.createdAt` added - a DB-authoritative row write time, distinct from the runtime-clock `startedAt`. (`AgentEvidence` already had `createdAt`.)
- `AgentRun.resumeState?: Readonly<Record<string, unknown>> | null` added - the opaque checkpoint the A4 resumable `tick()` runtime reads to pick up a lost serverless invocation. **Resumability is a property of the RUN**, independent of any tool's `executionMode` (per the G02 architectural lock).
- `AgentRun.deletedAt?` added - only `AgentRun` is soft-deletable.

Per AN1.2 ("a new optional field ... does not bump" the contract version), these are additive on types with zero persisted instances.

2.2 Field -> contract map

Prisma field	AF-v1 contract field	SQL type
<code>AgentRun.id/agentId/agentVersion/userId</code>	<code>runId</code>	TEXT NOT NULL
<code>AgentRun.status</code>	<code>AgentRun.status:</code> <code>AgentRunStatus</code>	"AgentRunStatus" NOT NULL DEFAULT 'queued'
<code>AgentRun.trigger</code>	<code>AgentRun.trigger:</code> <code>AgentRunTrigger</code>	"AgentRunTrigger" NOT NULL DEFAULT 'manual'
<code>AgentRun.input</code>	<code>input: unknown</code>	JSONB NOT NULL
<code>AgentRun.plan/output</code>	<code>plan?/output?: unknown</code>	JSONB (nullable)
<code>AgentRun.limits</code>	<code>limits: AgentRunLimits</code>	JSONB NOT NULL (snapshot at creation)
<code>AgentRun.creditsEstimated/creditsConsumed</code>	<code>consumption</code>	DOUBLE PRECISION NOT NULL DEFAULT 0
<code>AgentRun.metadata</code>	<code>metadata:</code> <code>Record<string, unknown></code>	JSONB NOT NULL DEFAULT '{}'
<code>AgentRun.resumeState</code>	<code>resumeState?</code>	JSONB (nullable)
<code>AgentRun.startedAt/completedAt/deletedAt</code>	<code>start string \ null</code>	TIMESTAMP(3) (nullable)
<code>AgentRun.createdAt/updatedAt</code>	same	TIMESTAMP(3) (updatedAt = @updatedAt)
<code>AgentStep.index</code>	<code>index: number</code> (0-based, gapless)	INTEGER NOT NULL, unique w/ runId
<code>AgentStep.kind/status</code>	<code>AgentStepKind/AgentStepStatus</code>	enum NOT NULL
<code>AgentStep.summary</code>	<code>summary: string</code>	TEXT NOT NULL
<code>AgentStep.input/output</code>	<code>input?/output?: unknown</code>	JSONB (nullable)
<code>AgentStep.startedAt/completedAt</code>	same	TIMESTAMP(3) NOT NULL

Prisma field	AF-v1 contract field	SQL type
AgentStep.durationMs	number	INTEGER NOT NULL
AgentStep.creditsConsumed	number	DOUBLE PRECISION NOT NULL DEFAULT 0
AgentToolCall.toolId/toolVersion	same	TEXT NOT NULL
AgentToolCall.status	status: ToolResultStatus	"AgentToolCallStatus" NOT NULL
AgentToolCall.permissionChecked	string[]	TEXT[] DEFAULT ARRAY[]::TEXT[]
AgentToolCall.evidenceIds	string[]	TEXT[] DEFAULT ARRAY[]::TEXT[]
AgentToolCall.creditCost	number	DOUBLE PRECISION NOT NULL DEFAULT 0
AgentEvidence.type	type: AgentEvidenceType	"AgentEvidenceType" NOT NULL
AgentEvidence.claim/source/sourceId	same	TEXT NOT NULL
AgentEvidence.timestamp	timestamp: string (fact-true time)	TIMESTAMP(3) NOT NULL
AgentEvidence.data/provenance	data: unknown / provenance: EvidenceProvenance	JSONB NOT NULL
AgentEvidence.relevance/confidence	number (0..1)	DOUBLE PRECISION NOT NULL
AgentEvidence.toolCallId	toolCallId?: string	TEXT (nullable FK)

The generated Prisma model types are asserted at compile time to carry every one of these fields (validate-agent-run-persistence.ts, the `_*HasContractFields` mapped types - tsc fails if any is missing).

3. Generated migration

- **Name:** 20260906120000_add_agent_runtime_persistence
- **Generated by:** prisma migrate diff --from-schema <pre-A3 schema> --to-schema prisma/schema.prisma --script (offline; no DB, no shadow DB)
- **Size:** 6 CREATE TYPE, 4 CREATE TABLE, 25 CREATE INDEX (1 unique), 6 ADD CONSTRAINT ... FOREIGN KEY
- **Header** states STATUS: NOT APPLIED and the migrate dev prohibition.

3.1 SQL review - findings

Check	Result
DROP anywhere	none
ALTER TABLE on an existing table	none - every ALTER TABLE targets one of the 4 new tables and is an ADD CONSTRAINT <code>"*_fkey" FOREIGN KEY</code>
Reference to "Agent" / "AgentTask" / "AgentMemory" / "AgentActivity" / "User"	none
Enum value spelling vs AF-v1 constants	exact match (asserted: <code>Object.values(PrismaEnum) === AF-v1 array, per enum</code>)
AgentStep uniqueness	CREATE UNIQUE INDEX "AgentStep_runId_index_key" ON "AgentStep"("runId", "index") - enforces gapless 0-based ordering at the DB
FK delete behaviour	all 6 FKs are ON DELETE CASCADE - a run's entire subtree (steps -> tool calls -> evidence) is one atomic unit; hard-deleting a run removes its whole history, and nothing is ever orphaned

Check	Result
Append-only child tables	AgentStep / AgentToolCall / AgentEvidence have no updatedAt, no deletedAt column
Mutable run table	AgentRun has updatedAt (@updatedAt) + deletedAt + resumeState
Secrets in schema	none - provenance / metadata / data are JSONB written by the runtime; the AF-v1 EvidenceProvenance contract already forbids keys/tokens/raw payloads
Default-heavy inserts	status, trigger, creditsEstimated, creditsConsumed, metadata, permissionChecked, evidenceIds, all createdAt have safe defaults; required-no-default fields (input, limits, evidence scalars) are always supplied by the runtime

3.2 Relationship analysis

AgentRun (1) --< AgentStep (N)	FK AgentStep.runId	ON DELETE CASCADE
AgentRun (1) --< AgentToolCall (N)	FK AgentToolCall.runId	ON DELETE CASCADE
AgentStep (1) --< AgentToolCall (N)	FK AgentToolCall.stepId	ON DELETE CASCADE
AgentRun (1) --< AgentEvidence (N)	FK AgentEvidence.runId	ON DELETE CASCADE
AgentStep (1) --< AgentEvidence (N)	FK AgentEvidence.stepId	ON DELETE CASCADE
AgentToolCall (0..1) --< AgentEvidence (N)	FK AgentEvidence.toolCallId (nullable)	ON DELETE CASCADE

- AgentToolCall carries **both** runId and stepId - so "every tool call in this run" is a single indexed query without a join through AgentStep, and the step linkage is still explicit.
- AgentEvidence carries runId + stepId always, toolCallId when a tool produced it. This is the table that answers **"why did this agent produce this conclusion"** - `SELECT * FROM AgentEvidence WHERE runId = ?` reconstructs the full evidence basis with no log replay.
- The full trace tree (AgentRunTrace read model in the contract) is AgentRun + its steps (ordered by index) + toolCalls + evidence.

3.3 Index analysis

Index	Serves
AgentRun(userId)	"my runs" list
AgentRun(agentId), AgentRun(agentId, createdAt)	one agent's run history, newest-first
AgentRun(status)	admin/ops filters
AgentRun(status, updatedAt)	the A4 tick() scheduler - <code>WHERE status IN (running, planning, awaiting_approval) ORDER BY updatedAt</code> to find the next resumable run
AgentRun(createdAt), AgentRun(deletedAt)	time scans, soft-delete filter
AgentStep(runId), unique AgentStep(runId, index)	ordered trace reconstruction + gapless-ordering guarantee
AgentStep(kind)	"all tool_call steps", evaluation queries
AgentToolCall(runId), (stepId), (toolId)	run trace, step drill-down, per-tool usage/cost analytics
AgentEvidence(runId), (stepId), (toolCallId), (type)	evidence basis reconstruction, per-type evidence queries

3.4 Uniqueness / cascade semantics

- Only one uniqueness constraint:** (runId, index) on AgentStep. It makes the append-only step log tamper-evident against a double-write at the same position (two concurrent tick() invocations cannot both claim step n).

- No unique constraint on `AgentToolCall` / `AgentEvidence` - multiple tool calls or evidence rows per step are legitimate.
- Cascade is delete-only; there is **no update-cascade concern** because the parent keys (`cuid()` ids) are immutable.

4. Compatibility analysis

Concern	Finding
Existing tables	Zero schema change. Migration SQL touches no existing table.
Seeded Agent rows (<code>prisma/seed.ts</code>)	Unaffected - no FK from <code>AgentRun</code> to <code>Agent</code> , <code>agentId</code> is a free string. A seeded agent's id can be used as <code>AgentRun.agentId</code> with no constraint.
Legacy <code>/dashboard/agents + services/agents/* + GET /api/private/agents</code>	Unaffected - they never touch the new tables.
<code>prisma generate</code>	Succeeds; new model + enum types emitted to <code>lib/generated/prisma/</code> .
<code>prisma validate</code>	The schema is valid.
<code>prisma format</code>	Idempotent (schema already formatted).
pgvector extension	Migration adds no extension, no vector column; <code>CREATE TYPE/CREATE TABLE</code> only. Safe to apply with <code>migrate deploy</code> ; never <code>migrate dev</code> .
Rollback	Trivial - <code>DROP TABLE</code> the 4 tables + <code>DROP TYPE</code> the 6 enums; nothing else to restore.
Data backfill	None - all-new tables, no existing rows to migrate.

5. Validation tests

`npm run validate:agent-run-persistence` - **17 passed, 0 failed** (offline, no DB):

- generated `AgentRunStatus` / `AgentRunTrigger` / `AgentStepKind` / `AgentStepStatus` / `AgentToolCallStatus` / `AgentEvidenceType` enums each **equal** the corresponding AF-v1 vocabulary
- migration file present; header marks it `NOT APPLIED`; forbids `migrate dev`
- migration creates **exactly** the 6 enums and 4 tables - no more
- migration is **purely additive** - no `DROP`, no `ALTER` on any existing table, every `ALTER` is an `ADD ... FOREIGN KEY` on a new table
- migration **never** names a legacy table
- (`runId`, `index`) uniqueness present on `AgentStep`
- all **6** FKs are `ON DELETE CASCADE`
- hot-query index coverage present (incl. `AgentRun(status, updatedAt)` for the `tick()` scheduler and `AgentEvidence(runId)` for conclusion tracing)
- `AgentStep` / `AgentToolCall` / `AgentEvidence` carry no `updatedAt/deletedAt`; `AgentRun` carries `updatedAt + deletedAt + resumeState`
- migration status asserted `GENERATED` / `REVIEWED` / `NOT APPLIED`

Regression: `validate:agent-contracts` -> **38/0**, `validate:agent-tools` -> **20/0** (22/0 with `RUN_LIVE_AGENT_TOOLS=1`).

6. TypeScript result

`npx tsc --noEmit: 0 errors` in `types/agent-framework/**`, `services/agent-framework/**`, `scripts/validate-agent-*.ts`, `lib/generated/prisma/**` (new models). The `*HasContractFields` compile assertions in the validation script pass - the generated Prisma types carry every AF-v1 field.

Repo-wide: 77 errors, **all** in the stale generated `.next/dev/types/validator.ts` (gitignored; regenerated by next build). Pre-existing, unrelated to A3.

7. Files changed

Added (3):

```
frontend/prisma/migrations/20260906120000_add_agent_runtime_persistence/migration.sql  (GENERATED + REVIEWED)
frontend/scripts/validate-agent-run-persistence.ts
frontend/docs/architecture/AN1.5-agent-run-persistence.md  (+ .pdf)
```

Modified (3):

```
frontend/prisma/schema.prisma          + 6 enums, 4 models (appended; no existing line c
frontend/types/agent-framework/agent-run-contract.ts  + AgentStep.createdAt, AgentToolCall.createdAt, A
frontend/package.json                  + "validate:agent-run-persistence"
```

`frontend/lib/generated/prisma/**` is regenerated locally by `prisma generate` but is **gitignored** (the repo regenerates it on `postinstall/build`), so it is not part of this commit.

Not touched: every existing model, `Agent/AgentTask/AgentMemory/AgentActivity`, the legacy agents UI + API, every other route/service.

8. Stopping point

A3 stops here. The migration is generated and SQL-reviewed. It is **not applied** and no runtime code reads or writes these tables yet (that is A4).

G03 review requested: `schema + migration.sql`. On acceptance -> authorize applying the migration, then A4 (server-side resumable `tick()` runtime + `LimitEnforcer` + `RunTracer`).

End AN1.5.